Interrupt-Guided Neural Code Completion with Static Analysis of Dependencies

Master Colloquium

Niklas Henrik Loeser | 3737339

2024-10-14

Heidelberg University
Institute of Computer Science
Artificial Intelligence for Programming (AIP)

Prof. Dr. Artur Andrzejak

Contents

1. Introduction 3
2. Comparison to Related Work 11
3. Interrupt-Based Meta-Strategy 18
4. Full Approach 25
5. Results 28
6. One-Click-Demo 38
7. Discussion 39
8. Appendix 76

1. Introduction

1.1 Motivation

- Causal Language Models (CLMs) are used for code completion
- CLMs help reduce writing boilerplate code (e.g. Copilot)
- CLMs trained on data *until cut-off date*
- CLMs have issues with changing dependencies

1.2 Example Pydantic 1 - Copilot

```python
from typing import Any, Dict
from pydantic import BaseModel

class User(BaseModel):
    ...

def convert_user_to_dict(user: User) -> Dict[str, Any]:
    """Convert the user into a Python dictionary.
    ...
    """
    # the function user.dict is deprecated, and user.model_dump should be used instead
    return user.dict()
```

1.3 Example Pydantic 1 - Solution

```python
from typing import Any, Dict
from pydantic import BaseModel

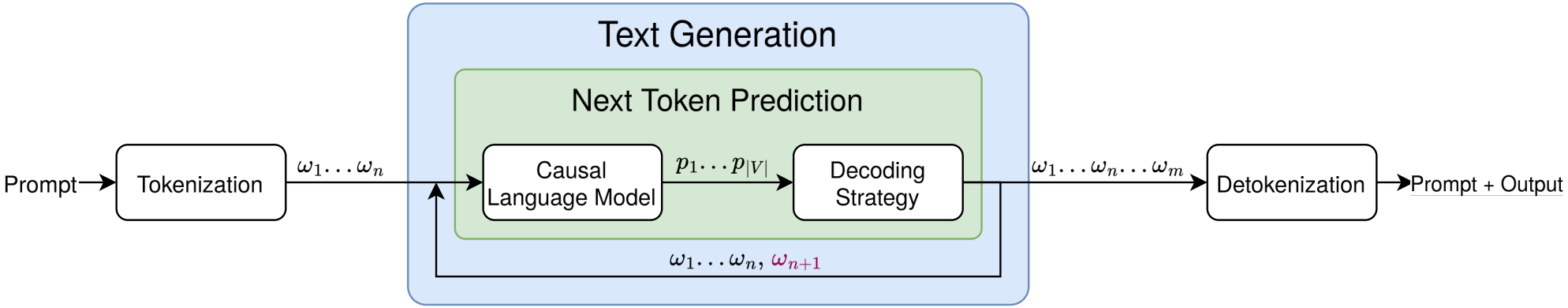
class User(BaseModel):
    ...

def convert_user_to_dict(user: User) -> Dict[str, Any]:
    """Convert the user into a Python dictionary.
    ...
    """
    return user.model_dump()
```

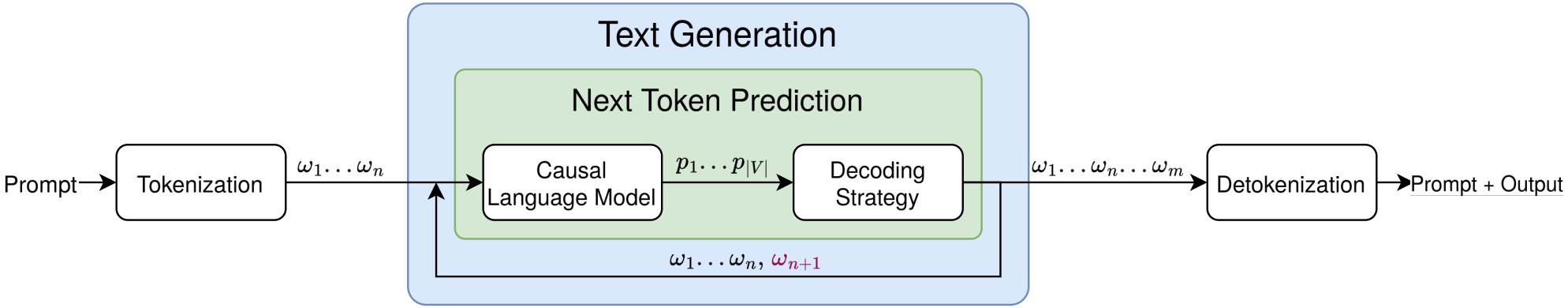
1.4 Problem Scenarios

1. *Feature recently replaced*
2. Feature recently removed
3. Feature recently added
4. Feature changed in any way, but not represented in training data

1.5 Background - Causal Language Models



1.5 Background - Causal Language Models



- V: vocabulary of all tokens
- token: subword, word, or character
- $\omega_1 \dots \omega_n; \omega \in V$: sequence of tokens ($\Longrightarrow$ text)
- $p_1 \dots p_{|V|}; \sum_{i=1}^{|V|} p_i = 1$: probability distribution over V

1.6 Goals

1. Retrieval: Retrieve accurate code context using IDE tooling
2. Injection: Inject context into model, to guide it …
   - … away from incorrect suggestions (deprecated / removed features)
   - … towards correct suggestions (new features)
3. Low performance overhead

1.7 Contributions

1. Static Analysis-guided neural code completion
   - Online monitoring of code completion
   - Accurate context retrieval
   - Hybrid injection of context using Monitor-Guided-Decoding (MGD)
2.
3.

1.7 Contributions

1. Static Analysis-guided neural code completion
   - Online monitoring of code completion
   - Accurate context retrieval
   - Hybrid injection of context using Monitor-Guided-Decoding (MGD)

2. Interrupt-Based Meta-Strategy (IBMS)
   - Inject context into the prompt “during” generation
   - $\Longrightarrow$ Low performance overhead
   - Applicable outside of code completion

3.

1.7 Contributions

1. Static Analysis-guided neural code completion
   - Online monitoring of code completion
   - Accurate context retrieval
   - Hybrid injection of context using Monitor-Guided-Decoding (MGD)

2. Interrupt-Based Meta-Strategy (IBMS)
   - Inject context into the prompt “during” generation
   - $\Longrightarrow$ Low performance overhead
   - Applicable outside of code completion

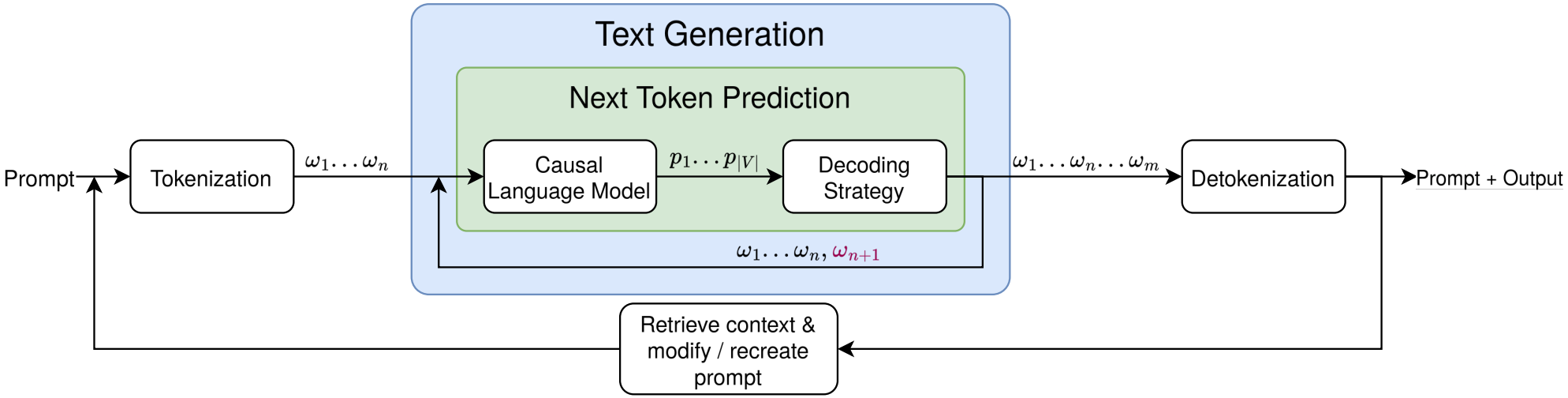
3. DependencyEval dataset
   - Handcrafted dataset to evaluate the approach used by the model
   - 25 Python tasks with fully pinned code environments
   - Evaluation framework to evaluate approach

2. Comparison to Related Work

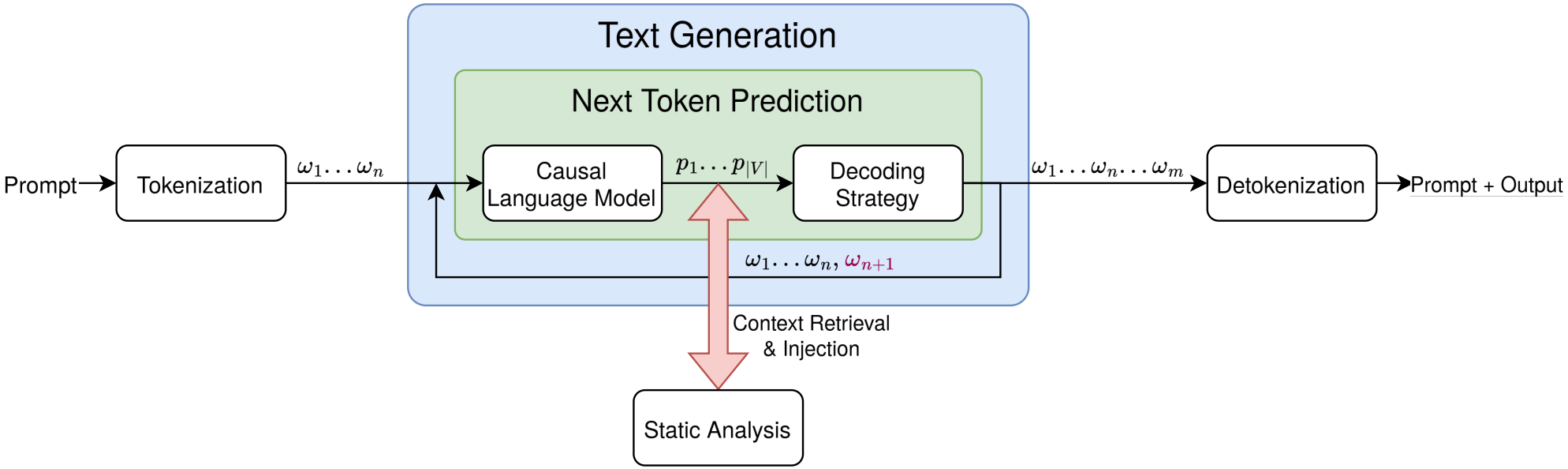
2.1 Categories of Related Work

1. Prompt Injection Works
   - Single-step meta-strategies
   - Multi-step meta-strategies
2. Logit-based Approaches

2.2 Prompt Injection Works



2.3 Logit-based Approaches



2.4 DocPrompting [1]

- Key points
  - Single-step meta-strategy
  - Context source: Online Documentation (DevDocs)
  - Retrieval process: Embedding search
  - Injection point: Once before generation into prompt
-
-

2.4 DocPrompting [1]

- Key points
  - Single-step meta-strategy
  - Context source: Online Documentation (DevDocs)
  - Retrieval process: Embedding search
  - Injection point: Once before generation into prompt
- Benefits
  - Flexible which documentation to embed
  - Documentation may include high quality user-submitted guides / examples
  - Fast
-

2.4 DocPrompting [1]

- Key points
  - Single-step meta-strategy
  - Context source: Online Documentation (DevDocs)
  - Retrieval process: Embedding search
  - Injection point: Once before generation into prompt
- Benefits
  - Flexible which documentation to embed
  - Documentation may include high quality user-submitted guides / examples
  - Fast
- Limitations
  - Context retrieved based on given code, *not to-be-generated code*
  - No differentiation between dependency sources

2.5 RepoCoder [2]

- Key points
  - Multi-step meta-strategy
  - Context source: Repository excluding environment files (venv, node_modules, etc.)
  - Retrieval process: Embedding search
  - Injection point: Once before generation into prompt
-
-

2.5 RepoCoder [2]

- Key points
  - Multi-step meta-strategy
  - Context source: Repository excluding environment files (venv, node_modules, etc.)
  - Retrieval process: Embedding search
  - Injection point: Once before generation into prompt
- Benefits
  - Full source code as context, including docstrings
  - Bases retrieval on generated code
-

2.5 RepoCoder [2]

- Key points
  - Multi-step meta-strategy
  - Context source: Repository excluding environment files (venv, node_modules, etc.)
  - Retrieval process: Embedding search
  - Injection point: Once before generation into prompt
- Benefits
  - Full source code as context, including docstrings
  - Bases retrieval on generated code
- Limitations
  - Slower due to multiple steps
  - No retrieval from dependencies

2.6 Monitor-Guided-Decoding[1] [3]

- Key points
  - Online monitoring / manipulation of token sequence
  - Context source: IDE completion entries
  - Retrieval process: Request language server
  - Injection point: Next token probability distribution
-
-

2.6 Monitor-Guided-Decoding[1] [3]

- Key points
  - Online monitoring / manipulation of token sequence
  - Context source: IDE completion entries
  - Retrieval process: Request language server
  - Injection point: Next token probability distribution
- Benefits
  - Accurate context retrieval
  - Low performance overhead
-

2.6 Monitor-Guided-Decoding[1] [3]

- Key points
  - Online monitoring / manipulation of token sequence
  - Context source: IDE completion entries
  - Retrieval process: Request language server
  - Injection point: Next token probability distribution
- Benefits
  - Accurate context retrieval
  - Low performance overhead
- Limitations
  - Injection of unstructured context (docstring) hard
  - Only focuses on available code, should disallow deprecated code

3. Interrupt-Based Meta-Strategy

3.1 Goals

- Online injection of context into the prompt (e.g. docstrings)
- ⟹ Enable MGD to inject unstructured context
- Full resumption of generation after injection

  (do not regenerate current partial result[1])
- ⟹ Support beam search decoding strategy

[1] *Details massively simplified here*

3.2 Approach

- Add special “interruption” token to model / tokenizer
- Stop generation on interrupt token
- MGD sets interrupt token when context available
- Edit prompt with provided context (*location* / *scope*)
- Patched model generation to enable full resumption[1]

[1] *To fully enable beam search decoding*

3.3 Example

```python
from typing import Any, Dict
from pydantic import BaseModel

class User(BaseModel):
    ...

def convert_user_to_dict(user: User) -> Dict[str, Any]:
    """Convert the user into a Python dictionary.
    ...
    """
    return user.
```

3.3 Example

```python
from typing import Any, Dict
from pydantic import BaseModel

class User(BaseModel):
    ...

def convert_user_to_dict(user: User) -> Dict[str, Any]:
    """Convert the user into a Python dictionary.
    ...
    """
    # The following methods are deprecated, use alternatives as instructed in the given docstrings:
    # - The `dict` method is deprecated; use `model_dump` instead.
    # - The `json` method is deprecated; use `model_dump_json` instead.
    # - ...
    return user.
```

3.3 Example

Python

```python
from typing import Any, Dict
from pydantic import BaseModel

class User(BaseModel):
    ...

def convert_user_to_dict(user: User) -> Dict[str, Any]:
    """Convert the user into a Python dictionary.
    ...
    """
    return user.model_dump()
```

3.4 Refresher: Key points

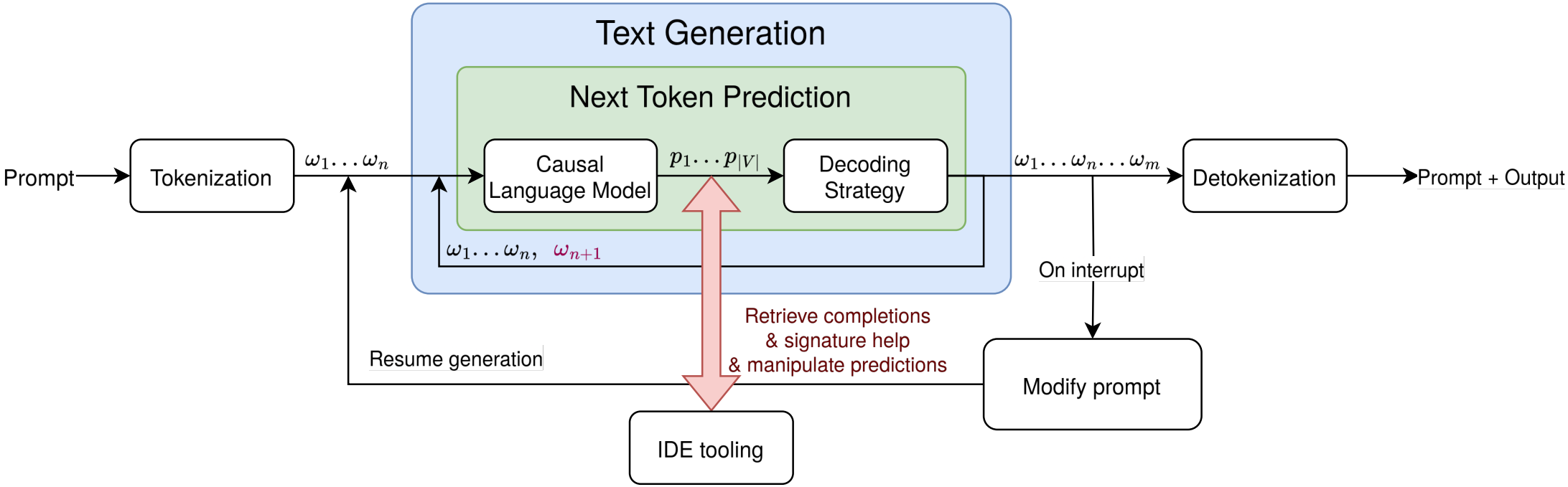
- Inject context into the prompt “during” generation
- $\Longrightarrow$ Low performance overhead
- Applicable outside of code completion

4. Full Approach

4.1 Key points

- Hybrid approach using MGD and IBMS
- Retrieve context from IDE tooling in real-time:
  - Code completion entries
  - Signature help
- Downrank deprecated code completions
- Uprank other code completions
- IBMS is main tool to guide model, prediction manipulation secondary

4.2 Overview


Text Generation
Next Token Prediction
Prompt
Tokenization
$\omega_1 \dots \omega_n$
Causal Language Model
$p_1 \dots p_{|V|}$
Decoding Strategy
$\omega_1 \dots \omega_n \dots \omega_m$
Detokenization
Prompt + Output
$\omega_1 \dots \omega_n$, ω_{n+1}
On interrupt
Retrieve completions & signature help & manipulate predictions
Resume generation
Modify prompt
IDE tooling

5. Results

5.1 DependencyEval Dataset

- Handcrafted dataset to evaluate the approach used by the model
- 25 Python tasks with fully pinned code environments
- Evaluation framework to evaluate approach
- Captures tasks for different problem scenarios:
  - Feature recently *replaced*
  - Feature recently added
  - Feature recently removed
  - Feature changed in any way, but not represented in training data

5.1 DependencyEval Dataset

JSON

```json
{
  "task_name": "textual_1",
  "package_dependencies": [
    "textual==0.51.0"
  ],
  "reason": "The parameter tab_behavior was renamed",
  "kind": "modification",
  "date": "2024-02-15",
  "code_kind": "parameter",
  "modification_kind": "rename",
  "changelog": "https://github.com/Textualize/textual/blob/v0.51.0/CHANGELOG.md",
  "python_version": "3.8"
}
```

5.2 DependencyEval Dataset - Tests

1. Output Correctness Test
   - Check the output of the function
   - Check set attributes, etc.
2. Approach Correctness Test
   - Check used variables *during* the function call

5.3 Experiment Environment

Component	Value
GPU	4x NVIDIA GeForce RTX 3090
OS	Ubuntu 21.04
Docker	20.10.7

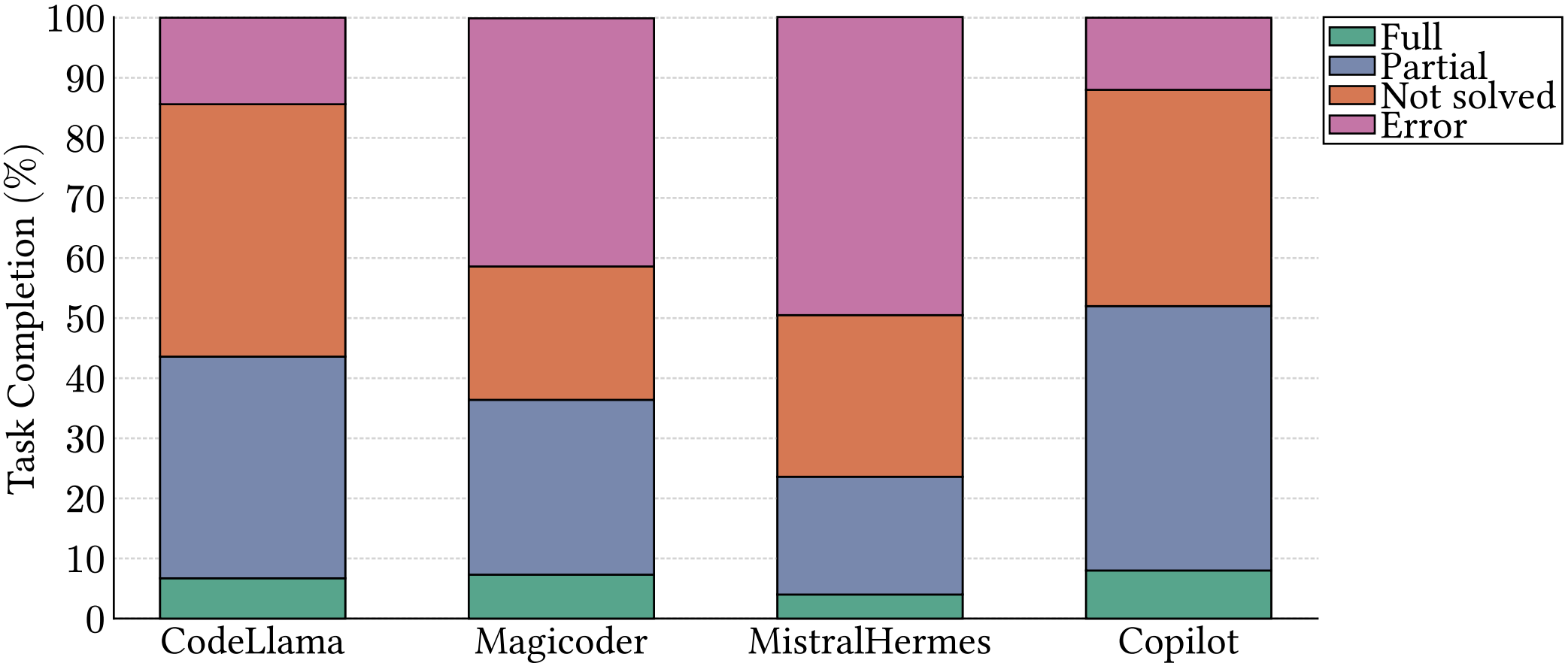
5.4 Experiment Setup

1. Models:
   - Code Llama Instruct 7B (CodeLlama)
   - Magicoder-S-DS 6.7B (Magicoder)
   - MistalHermes-CodePro 7B (MistalHermes)
   - Copilot (only for baseline)
   - *(each for 6 different decoding strategies)*
2. Baseline: Default model generation
3. Approach: Use one of the three configurations:
   - Default
   - Masked
   - Use completion

5.5 Result Labels

1. Full / Fully solved
   - *both* unit tests pass (functional and approach correctness)
2. Partial / Partially solved
   - only *one* of the unit tests pass
3. Not solved
   - neither of the unit tests pass
   - includes runtime exceptions, which the test framework catches
4. Error
   - evaluation cannot be made

5.6 Baseline Results - Models



5.7 Baseline Results - Task Scenarios

Name	Fully solved	Partially solved	Not solved	Error
pytorch_1	63.64%	12.73%	0%	23.64%
bidict_2	43.64%	20%	14.55%	21.82%
rich_2	20%	7.27%	40%	32.73%
pandas_1	14.55%	9.09%	16.36%	60%
dateutil_1	5.45%	32.73%	34.55%	27.27%
polars_1	3.64%	0%	65.45%	30.91%
pytorch_3	0%	72.73%	3.64%	23.64%
pytorch_2	0%	70.91%	0%	29.09%
dotted_1	0%	69.09%	5.45%	25.45%
pydantic_1	0%	67.27%	12.73%	20%
pydantic_3	0%	63.64%	10.91%	25.45%
sklearn_2	0%	56.36%	18.18%	25.45%
rich_1	0%	52.73%	25.45%	21.82%

Name	Fully solved	Partially solved	Not solved	Error
pydantic_2	0%	49.09%	23.64%	27.27%
numpy_1	0%	45.45%	7.27%	47.27%
textual_1	0%	30.91%	41.82%	27.27%
dotted_2	0%	25.45%	52.73%	21.82%
fastapi_1	0%	16.36%	41.82%	41.82%
bidict_1	0%	14.55%	52.73%	32.73%
emoji_1	0%	1.82%	74.55%	23.64%
textual_2	0%	1.82%	72.73%	25.45%
sqlalchemy_1	0%	0%	80%	20%
tsv2py_1	0%	0%	67.27%	32.73%
sklearn_1	0%	0%	0%	100%
theflow_1	0%	0%	0%	100%
Average	6.04%	28.8%	30.47%	34.69%

5.8 Approach Results - Task Scenarios

Name	Fully solved	Partially solved	Not solved	Error	Unchanged
pydantic_1	+44.44%	−51.85%	+1.85%	+5.56%	48.15%
textual_1	+25.93%	−31.48%	−24.07%	+29.63%	44.44%
pydantic_3	+25.93%	−44.44%	+12.96%	+5.56%	55.56%
rich_2	+24.07%	−3.7%	−37.04%	+16.67%	59.26%
dateutil_1	+14.81%	−1.85%	−27.78%	+14.81%	70.37%
pydantic_2	+9.26%	−31.48%	+7.41%	+14.81%	68.52%
pandas_1	+3.7%	+5.56%	±0%	−9.26%	90.74%
rich_1	+1.85%	−14.81%	−3.7%	+16.67%	81.48%
textual_2	±0%	+1.85%	−20.37%	+18.52%	79.63%
emoji_1	±0%	+1.85%	−24.07%	+22.22%	75.93%
sklearn_1	±0%	±0%	±0%	±0%	100%
theflow_1	±0%	±0%	±0%	±0%	100%
bidict_1	±0%	±0%	−7.41%	+7.41%	92.59%

Name	Fully solved	Partially solved	Not solved	Error	Unchanged
sqlalchemy_1	±0%	±0%	−9.26%	+9.26%	90.74%
tsv2py_1	±0%	±0%	−18.52%	+18.52%	81.48%
bidict_2	±0%	−7.41%	+1.85%	+5.56%	92.59%
dotted_2	±0%	−9.26%	−14.81%	+24.07%	75.93%
fastapi_1	±0%	−16.67%	−9.26%	+25.93%	74.07%
dotted_1	±0%	−20.37%	+1.85%	+18.52%	79.63%
numpy_1	±0%	−25.93%	+18.52%	+7.41%	74.07%
pytorch_2	±0%	−27.78%	+1.85%	+25.93%	72.22%
pytorch_3	±0%	−31.48%	+9.26%	+22.22%	68.52%
sklearn_2	±0%	−55.56%	+46.3%	+9.26%	44.44%
polars_1	−3.7%	±0%	−31.48%	+35.19%	64.81%
pytorch_1	−37.04%	+27.78%	±0%	+9.26%	62.96%
Average	+4.37%	−13.48%	−5.04%	+14.15%	73.93%

6. One-Click-Demo

7. Discussion

- Which assumptions does the approach hold about the model?
- Could the evaluation have included tasks of the standard library?
- What is the difference between multi-step meta-strategies and the IBMS?
- Which further enhancements would improve the approach?
- Which assumptions does the approach hold about the programming language?

7.1 Implementation difficulties

1. Model generating unwanted tokens (repetitions; descriptive sentences after generating code)
2. Runtime does not provide enough control over decoding internals
3. *Beam search*:
   - sequence reordering
   - separate state for different sequences
4. Detokenization not inverse of tokenization in practice
5. Model ignoring injected context

7.1 Implementation difficulties

6. Use of different virtual environments

7. Language servers not providing accurate deprecation information

7.2 Baseline Results - Best combinations

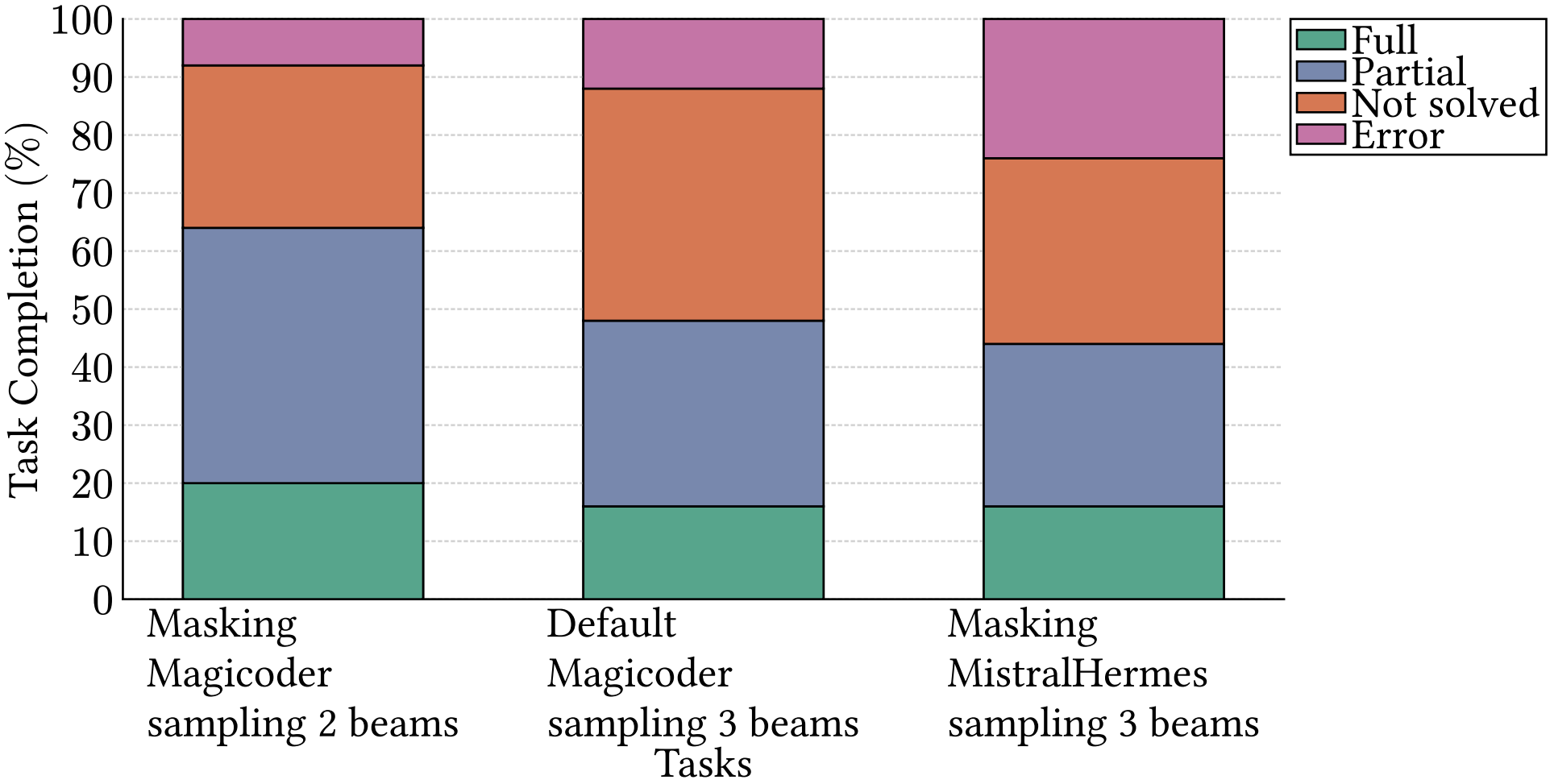


Figure 1: Overview over the top 3 baseline configurations.

7.3 Approach Results - Best combinations

Name	Fully solved	Partially solved	Not solved	Error	Unchanged
Default CodeLlama sampling 2 beams	+20%	−16%	−12%	+8%	72%
Masking MistralHermes sampling 2 beams	+16%	−8%	±0%	−8%	84%
Masking CodeLlama sampling 2 beams	+16%	−12%	−4%	±0%	84%

Table 1: Overview over the top 3 approach configurations.

7.4 Baseline Results - Worst combinations

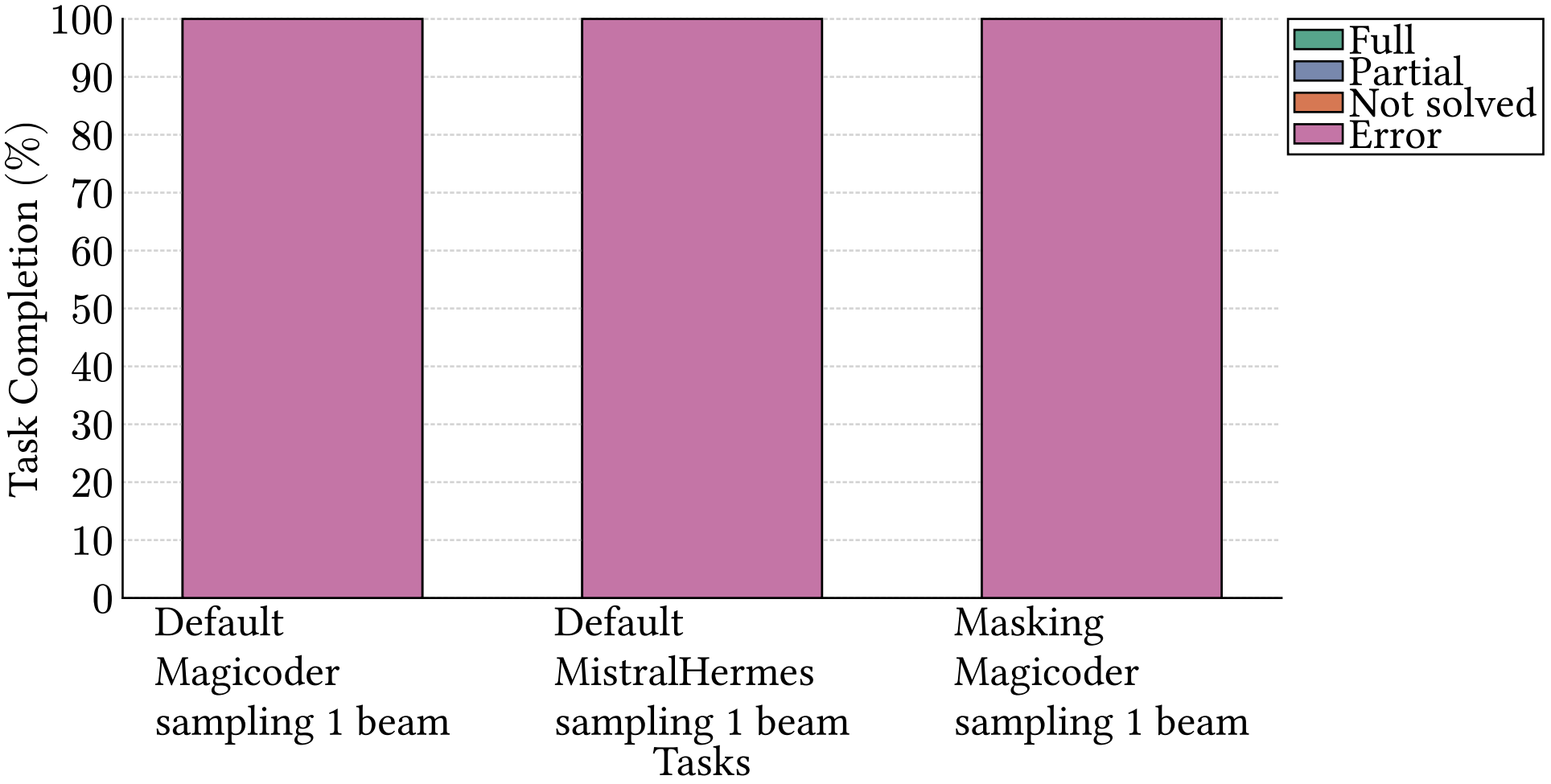


Figure 2: Overview over the top 3 baseline configurations.

7.5 Approach Results - Worst combinations

Name	Fully solved	Partially solved	Not solved	Error	Unchanged
Use Completions Magicoder greedy 3 beams	−8%	−48%	±0%	+56%	44%
Use Completions CodeLlama greedy 2 beams	−4%	−36%	±0%	+40%	60%
Use Completions MistralHermes greedy 2 beams	−4%	−20%	−32%	+56%	44%

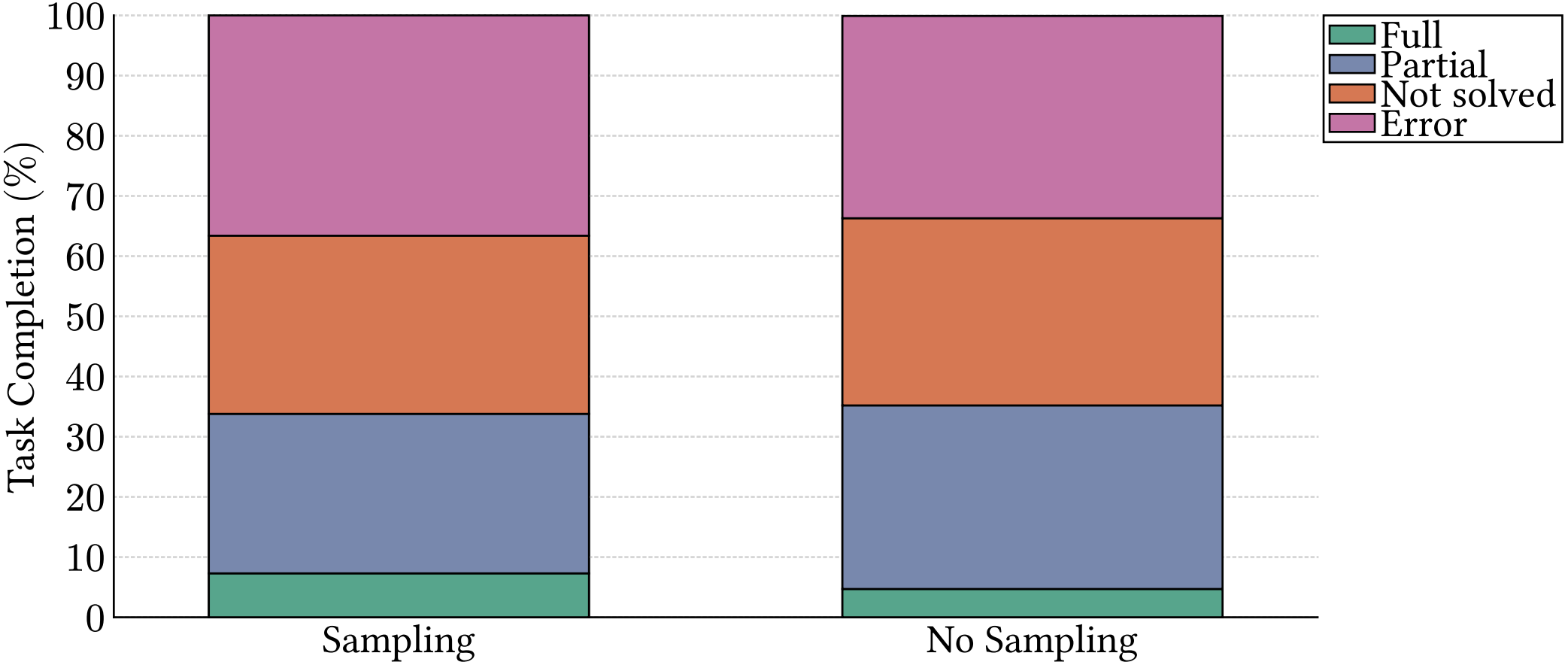
Table 2: Overview over the top 3 approach configurations.

7.6 Full Experiment Environment

Component	Value
CPU	2x AMD EPYC 7713 64-Core
Memory	1 TB
GPU	4x NVIDIA GeForce RTX 3090
OS	Ubuntu 21.04
Kernel	5.15.0
Docker	20.10.7
Python	3.9.18
Transformers	4.39.0
PyTorch	2.2.1
CUDA	11.8

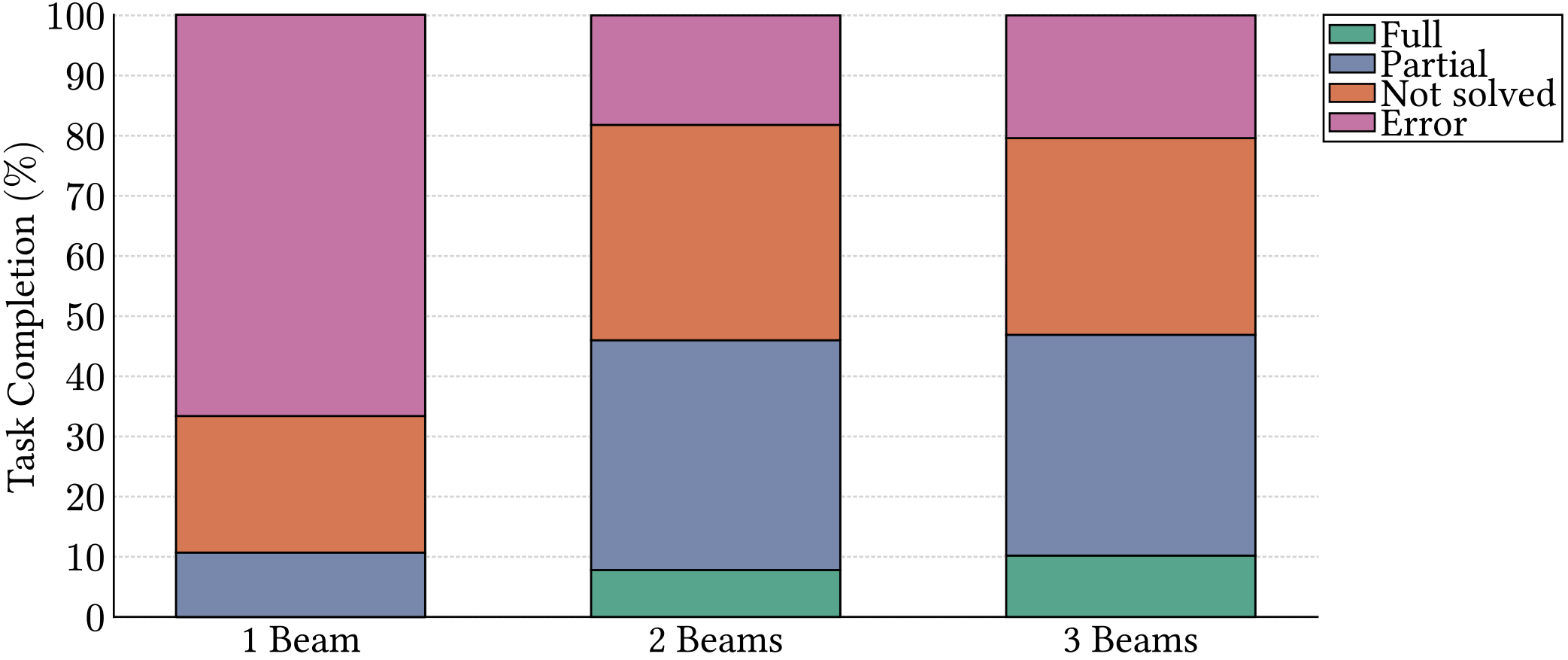
7.7 Baseline Results - Model configurations

Sampling



7.8 Baseline Results - Model configurations

Beam search



7.9 Tasks

```python
from typing import Any, Dict

from bidict import OnDup, OnDupAction, bidict


def insert_values_drop_old_on_dup(values: bidict, items: Dict[str, Any]):
    """Insert all key value pairs from items into values at once. Drop old keys and values on duplication.

    Args:
        values (bidict): Bidirectional mapping between keys and values
        items (Dict[str, Any]): Python mapping between keys and values to be inserted into values
    """
    values.putall(items, OnDup(key=OnDupAction.DROP_OLD, val=OnDupAction.DROP_OLD))
```

Listing 1: Task: bidict_1.py

7.9 Tasks

```python
from bidict import bidict


def invert_bidict_direction(values: bidict) -> bidict:
    """Return the inverse of the given bidirectional mapping instance.

    Args:
        values (bidict): Bidirectional mapping between any keys and values

    Returns:
        bidict: Inverse of values
    """
    return values.inverse
```

Listing 2: Task: bidict_2.py

7.9 Tasks

```python
from datetime import datetime

import dateutil


def current_datetime_in_local_timezone() -> datetime:
    """Return the current date and time in the local time zone.

    Returns:
        datetime: Current local date and time
    """
    return datetime.now(dateutil.tz.tzlocal())
```

Listing 3: Task: dateutil_1.py

7.9 Tasks

```python
from dotted.collection import DottedDict


def get_user_street_name(user: DottedDict) -> str:
    """Retrieve the street name of the user.

    Args:
        user (DottedDict): The user has the following JSON schema:
    {
        name: str,
        age: int,
        email: str,
        street: {
            number: int,
            name: str
        }
    }

    Returns:
        str: Street name
    """
    return user["street.name"]
```

7.9 Tasks

```python
from typing import Any

from dotted.collection import DottedList


def get_2d_board_entry(board: DottedList, index: str) -> Any:
    """Retrieve the value in the 2d board at the given index.

    Args:
        board (DottedList): A 2d dimensional board implemented through nested lists. The board may store any type of value.
        index (str): An index in the format of "column.row"

    Returns:
        Any: The value
    """
    return board[index]
```

Listing 5: Task: dotted_2.py

7.9 Tasks

```python
import emoji

THUMBS_UP = emoji.emojize(":thumbs_up:")
THUMBS_DOWN = emoji.emojize(":thumbs_down:")

def does_the_text_contain_only_emojis(text: str) -> str:
    """Return either thumbs up or down depending on text containing only emojis.

    Args:
        text (str): Any input text

    Returns:
        str: Thumbs up emoji if text only contains emojis. Else thumbs down.
    """
    return THUMBS_UP if emoji.purely_emoji(text) else THUMBS_DOWN
```

Python

Listing 6: Task: emoji_1.py

7.9 Tasks

Python

```python
from contextlib import asynccontextmanager

from fastapi import FastAPI


def startup(app: FastAPI):
    app.state.startup = True

def shutdown(app: FastAPI):
    app.state.shutdown = True

def create_fastapi_app() -> FastAPI:
    """Create a new FastAPI app which calls the lifespan functions startup and shutdown.

    Returns:
        FastAPI: New FastAPI instance
    """
    @asynccontextmanager
    async def lifespan(app: FastAPI):
        startup(app)
        yield
        shutdown(app)
    return FastAPI(lifespan=lifespan)
```

7.9 Tasks

```python
import numpy as np
from typing import List

def add_strings_element_wise(a: List[str], b: List[str]) -> List[str]:
    """Add both lists of strings element-wise. Use the Numpy library.

    Args:
        a (List[str]): First list
        b (List[str]): Second list

    Returns:
        List[str]: Combined list
    """
    return np.strings.add(a, b)
```

Listing 8: Task: numpy_1.py

7.9 Tasks

```python
import pandas as pd

def get_first_group_entry_allow_na(grouped_df: pd.core.groupby.GroupBy) -> pd.core.generic.NDFrameT:
    """Return the first row for each group, while not skipping NA values.

    Args:
        grouped_df (pd.core.groupby.GroupBy): The already grouped data frame.

    Returns:
        pd.core.generic.NDFrameT: A generic multi dimensional dataframe, containing each first row result.
    """
    return grouped_df.first(skipna=False)
```

Listing 9: Task: pandas_1.py

7.9 Tasks

```python
from typing import List

import polars as pl


def lazy_filter_old_users(csv_file_path: str) -> List[str]:
    """Lazily return a list of all user names, which are older than 50. The name column is `name`, the age column is `age`.

    Args:
        csv_file_path (str): Path to a CSV file with input data

    Returns:
        List[str]: User names of users older than 50
    """
    df = pl.scan_csv(csv_file_path)
    names = df.filter(pl.col("age") > 50).select(pl.col("name")).collect()
    return names.to_series().to_list()
```

Listing 10: Task: polars_1.py

7.9 Tasks

```python
from typing import Any, Dict

from pydantic import BaseModel


class User(BaseModel):
    name: str
    email: str
    age: int

def convert_user_to_dict(user: User) -> Dict[str, Any]:
    """Convert the user into a Python dictionary.

    Args:
        user (User): Pydantic user model

    Returns:
        Dict[str, Any]: User attributes as a Python key value mapping
    """
    return user.model_dump()
```

Listing 11: Task: pydantic_1.py

7.9 Tasks

```python
from typing import Any, Dict

from pydantic import BaseModel


class User(BaseModel):
    name: str
    email: str
    age: int

def convert_user_to_json(user: User) -> str:
    """Convert the given user model into a JSON string.

    Args:
        user (User): Pydantic user model

    Returns:
        str: JSON string of user attributes
    """
    return user.model_dump_json()
```

Listing 12: Task: pydantic_2.py

7.9 Tasks

```python
from typing import Any, Dict

from pydantic import BaseModel


class User(BaseModel):
    name: str
    email: str
    age: int

def duplicate_user(user: User) -> User:
    """Duplicate the user.

    Args:
        user (User): The Pydantic user model

    Returns:
        User: Deep copy of the user
    """
    return user.model_copy()
```

Listing 13: Task: pydantic_3.py

7.9 Tasks

```python
from torch.nn import CrossEntropyLoss


def create_sum_cross_entropy_loss_module() -> CrossEntropyLoss:
    """Create an instance of CrossEntropyLoss which computes the sum of the cross entropy loss.

    Returns:
        CrossEntropyLoss: New instance which computes the sum of the cross entropy loss
    """
    return CrossEntropyLoss(reduction="sum")
```

Listing 14: Task: pytorch_1.py

7.9 Tasks

```python
from numbers import Number

import torch


def create_1d_tensor_in_range(start: Number, end: Number) -> torch.Tensor:
    """Return a 1d tensor with values from start to end.

    Args:
        start (Number): Starting number (inclusive)
        end (Number): End number (exclusive)

    Returns:
        torch.Tensor: Tensor with values from start to end
    """
    return torch.arange(start, end)
```

Listing 15: Task: pytorch_2.py

7.9 Tasks

```python
import torch


def calculate_cholesky(input: torch.Tensor) -> torch.Tensor:
    """Calculate the Cholesky decomposition.

    Args:
        input (torch.Tensor): Input tensor

    Returns:
        torch.Tensor: Cholesky decomposition of input tensor
    """
    return torch.linalg.cholesky(input)
```

Listing 16: Task: pytorch_3.py

7.9 Tasks

```python
from rich.style import Style


def clear_style(style: Style) -> Style:
    """Obtain a copy of style with all meta and links removed.

    Args:
        style (Style): target style

    Returns:
        Style: target style without meta and links
    """
    return style.clear_meta_and_links()
```

Listing 17: Task: rich_1.py

```python
from rich.prompt import Prompt


def create_case_insensitive_prompt(text: str) -> Prompt:
    """Create a prompt instance, providing the text and using case insensitivity.

    Args:
        text (str): prompt text

    Returns:
        Prompt: created prompt
    """
    return Prompt(text, case_sensitive=False)
```

Listing 18: Task: rich_2.py

```python
from sklearn.preprocessing import OneHotEncoder


def create_dense_one_hot_encoder() -> OneHotEncoder:
    """Create a OneHotEncoder which encodes categorical features into a dense matrix.

    Returns:
        OneHotEncoder: New instance of OneHotEncoder encoding categorical features into a dense matrix
    """
    return OneHotEncoder(sparse_output=False)
```

Listing 19: Task: sklearn_1.py

7.9 Tasks

```python
from sklearn.preprocessing import OneHotEncoder


def create_polars_compatible_one_hot_encoder() -> OneHotEncoder:
    """Create a OneHotEncoder which encodes categorical features into polars containers.

    Returns:
        OneHotEncoder: New instance of OneHotEncoder encoding categorical features into polars containers
    """
    encoder = OneHotEncoder()
    encoder.set_output(transform="polars")
    return encoder
```

Listing 20: Task: sklearn_2.py

7.9 Tasks

```python
from sqlalchemy import Row
from sqlalchemy.engine.row import _TP


def get_tuple_of_row(row: Row) -> _TP:
    """Return this row as a tuple.

    Args:
        row (Row): Input row

    Returns:
        _TP: Input row represented as a tuple
    """
    return row._tuple()
```

Listing 21: Task: sqlalchemy_1.py

7.9 Tasks

```python
from textual.widgets import TextArea


def create_textual_text_area_with_indent() -> TextArea:
    """Create a TextArea widget, which indents its content when tab is pressed.

    Returns:
        TextArea: New instance of TextArea with indentation on tab press
    """
    return TextArea(tab_behavior="indent")
```

Python

Listing 22: Task: textual_1.py

7.9 Tasks

```python
from textual.app import App
from textual.types import AnimationLevel


def create_app_without_animations() -> App:
    """Create a minimal textual App without animations.

    Returns:
        App: New App instance with disabled animations
    """
    app = App()
    app.animation_level = "none"
    return app
```

Listing 23: Task: textual_2.py

7.9 Tasks

```python
from theflow import Function


def square(x: int) -> int:
    return x*x

class MultiplyBy(Function):
    factor: int
    def run(self, y):
        return y*self.factor



class MultiplySquareFlow(Function):
    multiply: Function
    square: Function

    def run(self, x):
        y = self.multiply(x)
        y = self.square(y)
        return y
```

Listing 24: Task: theflow_1.py

7.9 Tasks

```python
def multiply_then_square(x: int, multiplication_factor: int) -> int:
    """Multiply x by multiplication factor, then square the result.

    Args:
        x (int): Input number
        multiplication_factor (int): Multiplication factor for x

    Returns:
        int: x times multiplication factor, then the squared result using the provided Functions
    """
    flow = MultiplySquareFlow(square=square,multiply=MultiplyBy(factor=multiplication_factor))
    return flow(x=x)
```

Listing 25: Task: theflow_1.py

7.9 Tasks

```python
from datetime import datetime
from typing import Any, List, Tuple

from tsv.helper import Parser


def parse_tsv_file(filename: str) -> List[Tuple[Any, ...]]:
    """The file at filepath contains entries in the tsv format. Parse the file into a Python list of tuples.

    Args:
        filename (str): Name of the TSV file. The TSV entries have the following columns: name, age, birthday

    Returns:
        List[Tuple[Any, ...]]: List of Python tuples of the tabular data
    """
    parser = Parser(fields=(str, int, datetime))
    with open(filename, "rb") as f:
        return parser.parse_file(f)
```

Listing 26: Task: tsv2py_1.py

8. Appendix

8.1 Acronyms

CLM: Causal Language Model

IBMS: Interrupt-Based Meta-Strategy

MGD: Monitor-Guided-Decoding

8.2 Bibliography

[1] S. Zhou, U. Alon, F. F. Xu, Z. Wang, Z. Jiang, and G. Neubig, “DocPrompting: Generating Code by Retrieving the Docs.” Accessed: Jun. 23, 2024. [Online]. Available: http://arxiv.org/abs/2207.05987

[2] F. Zhang *et al.*, “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation.” Accessed: Dec. 05, 2023. [Online]. Available: http://arxiv.org/abs/2303.12570

[3] L. A. Agrawal, A. Kanade, N. Goyal, S. K. Lahiri, and S. K. Rajamani, “Monitor-Guided Decoding of Code LMs with Static Analysis of Repository Context.”